

Embedded systems I / embedded programming

‘C’ Programming advanced

Pascal Sartoretti

v2.0

‘C’ : objectives

Be able to nice program in C language for embedded

List of chapters

- **Type casting**
- **C structures, unions, bitfields**
- **C pointers**
- **Functions pointers**
- **Stack, heap, malloc, free**
- **STM32 specifics**

TYPECAST in C

‘C’ : type casting

What is a type ?

Standard C types:

- **char, int, float, double**
- **Usage of <stdint.h>**

stdint.h

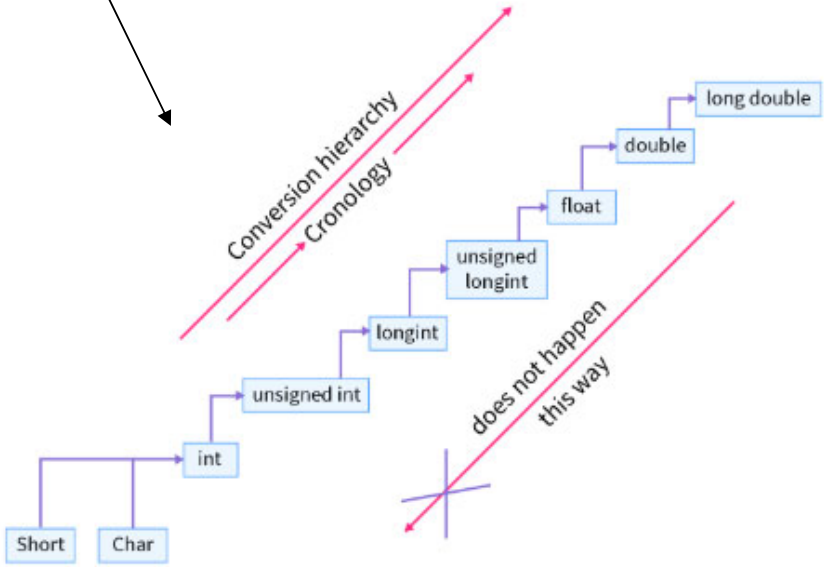
```
// standard C types
typedef unsigned char uint8_t;
typedef unsigned short uint16_t;
typedef unsigned long uint32_t;
```

‘C’ : type casting

Implicit or explicit ?

If not explicit, typecasting is implicit respecting this rule

```
void main(void)
{
  int8_t a = 120, b = -7;
  int16_t c, d;
 c = a;              // c = _______
 d = b;              // d = _______
 c = c * 3;          // c = _______
 a = c;              // a = _______
}
```



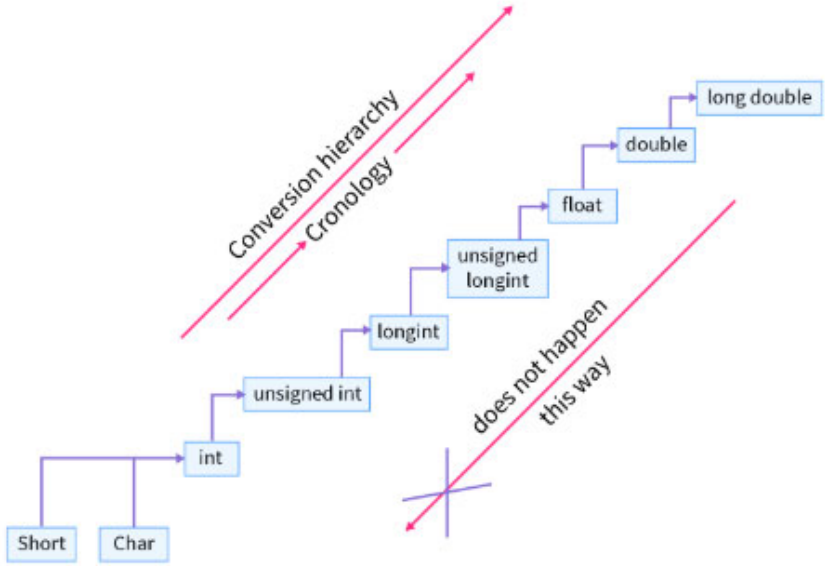
‘C’ : type casting

Implicit (automatic) !

Caution:

- **Implicit typecast is made with the bigger variable to the right of parenthesis**
- **Then, to the parenthesis**

```
// standard C types
void main(void)
{
  uint16_t a = 15, b = 2;
  float div;
  div = a / b;
  printf("div = %f\n",div);
}
```



Console >> div = _____

‘C’ : type casting

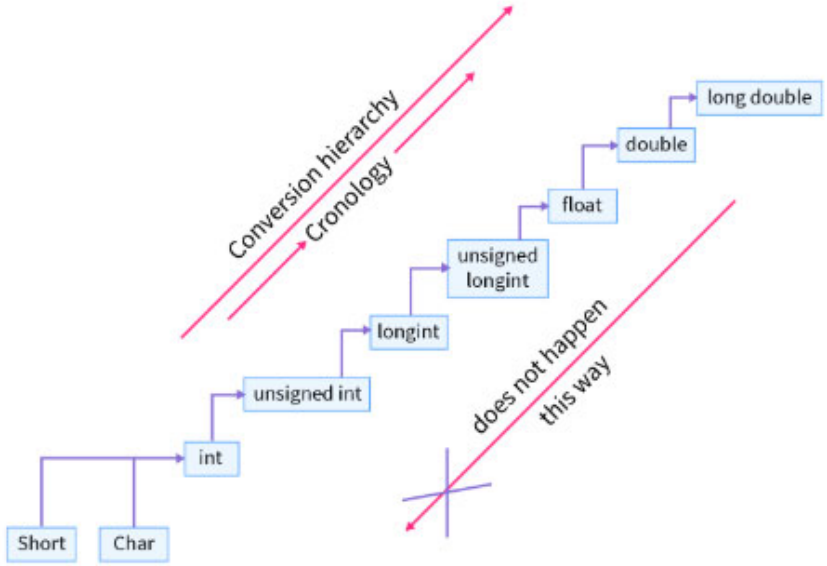
Explicit and implicit!

Caution:

- **Implicit typecast is made with the bigger variable to the right of parenthesis**
- **The bigger is now the float**

```
// standard C types
void main(void)
{
  uint16_t a = 15, b = 2;
  float div;
  div = (float)a / b;
  printf("div = %f\n",div);
}
```

Console >> div = _____

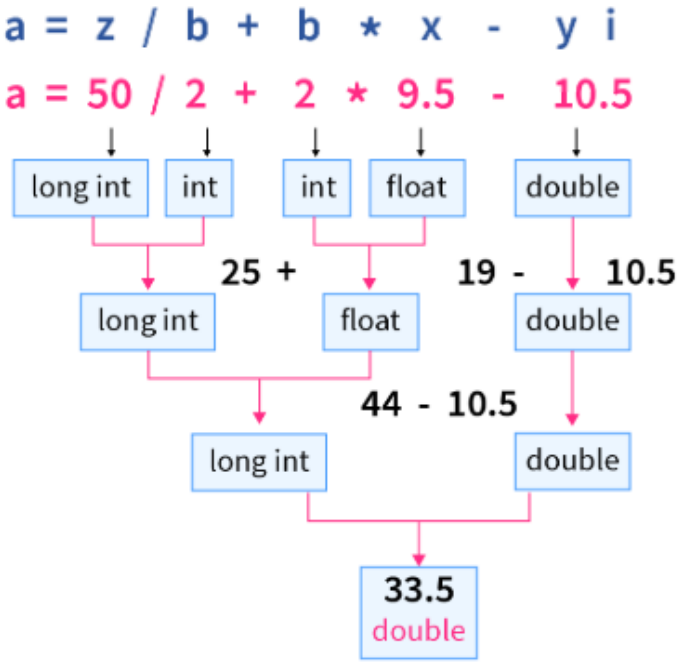


‘C’ : type casting

Prefer explicit vs implicit

```
// standard C types
void main(void)
{
  int16_t a,b = 2;
  float x = 9.5;
  double y = 10.5;
  uint32_t z = 50;

  a = z/b+b*x-y;
  printf("a = %f\n",a);
}
```



Caution: Left to right can cause a problem z=a/b*c

STRUCTURE in C

‘C’ : structures

What is an array?

An array allow grouping of elements of the same type

It may be useful to group elements of different types

```
...
uint16_t table[100];                // a table of 100 words
char hello[]= "My name is Bond !"; // a table of ASCII bytes
...
```



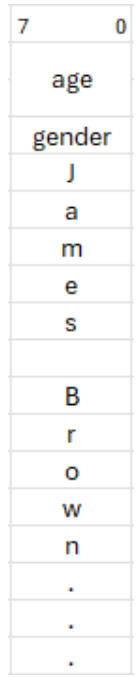
‘C’ : structures

Declaration of a structure without a typedef

A structure allow grouping of elements of different types

```
...
struct Student
{
  uint8_t age;                // how old is the student
  char gender;                // used for 'M' or 'F'
  char firstName[30];         // student first name
  struct Address address;     // previously defined structure
};

struct Student class[30];     // 30 student class
...
```



‘C’ : structures

Declaration with typedef

A structure allow grouping of elements of different types

```
...
typedef struct
{
  uint8_t age;                  // how old is the student
  char gender;                  // used for 'M' or 'F'
  char firstName[30];           // student first name
  Address address;              // previously defined structure
}Student;

Student class[30];    // 30 student class
...
```

‘C’ : structures

Function parameter 1/3

A structure is always passed by value




```
void clearStudent(Student theStudent)
{
  theStudent.age = 0;           // set to default value
  ...
}


void main(void)
{
  Student class[30];            // uninitialized variable
  clearStudent(class[0]);       // initialize it ?
  class[1] = class[0];          // copy it ?
  ...
}
```

‘C’ : structures

Function parameter 2/3

A structure must be passed as reference if we want to modify it



understand * and &

```
void clearStudent(Student * theStudent)
{
  theStudent->age = 0;           // set to default value
  ...
}

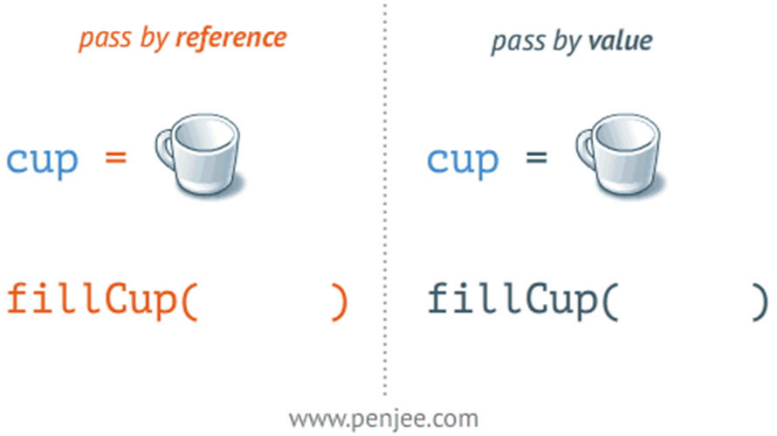

void main(void)
{
  Student class[8];              // uninitialized variable
  clearStudent(&class[0]);       // initialize it
  ...
}
```

‘C’ : structures

Function parameter 3/3

A structure can be passed as value or reference

```
...
clearStudent(&class[0]);      // pass by reference
...
```




```
...
clearStudent(class[0]);      // pass by value
...
```

‘C’ : structures – quiz time

Based on slide page 13, what is wrong below ?

```
1. class[2].gender = “M”;
2. class[5].age = 25;
3. class[30].gender = ‘F’;
4. class[0] = class[0] + 1;
5. class[3].age = 260;
6. class[4].firstName = class[5].firstName;
7. class[6] = class[3];
8. class[1].firstName = “James”;
```

UNION in C

‘C’ : union

What is a union?

It may be useful to access the same memory variable in different ways

The union offers this

```
...
union MyUnion
{
  uint8_t  bytesAccess[4];          // bytes access
  uint32_t wordAccess;              // 32 bits access on the same content
};
...
Union MyUnion myUnion;              // a variable of union type
myUnion.wordAccess = 0x12345678;    // one type of access
myUnion.bytesAccess[0] = 0xAA;      // another access
...
```

‘C’ : union

Declaration of a union without a typedef

A union allow accessing same memory value with different types

```
typedef struct
{
  uint8_t red;                  // red color of a pixel
  uint8_t green;                // green color of a pixel
  uint8_t blue;                 // blue color of a pixel
}Color;

union Pixel
{
  Color rgb;                    // struct access
  uint32_t mixColor;            // 32 bits access on the same content
};

union Pixel pixel;              // the pixel needs size of biggest element
...
```

‘C’ : union

Declaration with a typedef

A union allow accessing same memory value with different types

```
typedef struct
{
  uint8_t red;                  // red color of a pixel
  uint8_t green;                // green color of a pixel
  uint8_t blue;                 // blue color of a pixel
}Color;

typedef union
{
  Color rgb;                    // struct access
  uint32_t mixColor;            // 32 bits access on the same content
} Pixel;

Pixel pixel;                    // the pixel needs size of biggest element
```

‘C’ : union

Usage example

A union allow accessing same memory value with different types

```
Pixel pixel[20];

void main(void)
{
  pixel[0].rgb.blue = 0x80;              // blue color of a pixel is set to 50%
  pixel[1].rgb.red = pixel[0].rgb.blue; // red color of a pixel is a blue copy
  pixel[0].mixColor = 0xFF0000;          // full blue and others are off
};
```

‘C’ : union

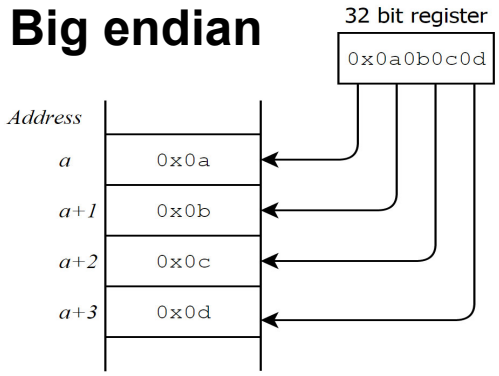
Memory position

```
pixel.mixColor = 0xA0B0C0D0;          // example on the STM32
```

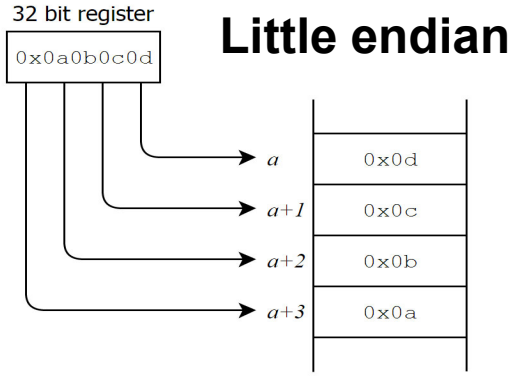
mixColor (uint32_t)

0x100	0x0D	red (uint8_t)
0x101	0x0C	green (uint8_t)
0x102	0x0B	blue (uint8_t)
0x103	0x0A	

Big endian



Little endian



‘C’ : union

Endianness caution

Some compilers include <endian.h>, but not all.

- **In this case, union will “portable” only if we define this BYTE_ORDER**

```
#if __BYTE_ORDER == __LITTLE_ENDIAN
typedef struct
{
  uint8_t dummy;            // one byte for align
  uint8_t blue;             // one byte blue
  uint8_t green;            // one byte green
  uint8_t red;              // one byte red
}Color;
#else
{
  uint8_t red;              // one byte red
  uint8_t green;            // one byte green
  uint8_t blue;             // one byte blue
}Color;
#endif
```

‘C’ : union

Dynamic endianness access

It’s always possible to know the endianness our processor uses !

```
uint32_t dummy;                 // any variable
uint32_t *p32dummy=&dummy;      // a 32 bits pointer
uint8_t  *p8ummy=&dummy;        // a 8 bits pointer
...
*p32Dummy = 0x12345678;         // write a know value
if(*p8Dummy == 0x78)            // check if LSB is at the lower address
{
    // processor is little endian
}
else
{
    // processor is big endian
}
```

Problem is we must write code for both access because it can’t be tested during compilation but during execution !

‘C’ : union

Endianness option

With gcc, the pragma scalar_storage_order offers the ability to choose the endianness

```
#pragma scalar_storage_order big-endian
typedef struct
{
  uint8_t red;                  // red color of a pixel
  uint8_t green;                // green color of a pixel
  uint8_t blue;                 // blue color of a pixel
}Color;
typedef union
{
  Color rgb;                    // struct access
  uint32_t mixColor;            // 32 bits access
} Pixel_be;

...
pixel_be.mixColor = 22;
```

	pixel_be	
red	0	0x100
green	0	0x101
blue	0	0x102
-	22	0x103

- **This option applies to struct and union, it means mixColor is bytes inverted**
- **The struct order is not changed !**
- **The full union has the same storage order !**

‘C’ : union

Endianness option

```
#pragma scalar_storage_order little-endian
typedef struct
{
  uint8_t red;                  // red color of a pixel
  uint8_t green;                // green color of a pixel
  uint8_t blue;                 // blue color of a pixel
}Color;
typedef union
{
  Color rgb;                    // struct access
  uint32_t mixColor;            // 32 bits access
} Pixel_le;
#pragma scalar_storage_order default

...
pixel_le.mixColor = 22;
```

	pixel_le	
red	22	0x100
green	0	0x101
blue	0	0x102
-	0	0x103

```
#pragma scalar_storage_order default  // returns to default compiler
```

‘C’ : union

Endianness option, typical complex application

Example of a pseudo Modbus frame:

- **Serial port write the data in the bytes part**
- **User want access the data logically with address, data, …**

Caution, the endianness of address and data is not the same as the endianness of crc !!!

	frame	
byte[0]	CMD	command
byte[1]	A_HH	address
byte[2]	A_H	
byte[3]	A_L	
byte[4]	A_LL	
byte[5]	D_H	data
byte[6]	D_L	
byte[7]	CRC_L	crc
byte[8]	CRC_H	

‘C’ : union

Endianness option, typical complex application

```
#pragma pack (1)
#pragma scalar_storage_order little-endian
typedef struct
{
  uint16_t crc;                 // crc is little endian
}Crc;
#pragma scalar_storage_order big-endian
typedef struct
{
  uint8_t command;              // 8 bits command type
  uint32_t address;             // address of register
  uint16_t data;                // data
  Crc crc;                      // crc
} Frame;
typedef union
{
  uint8_t bytes[9];             // serial received bytes
  Frame   frame;                // frame access
} TheFrame;
#pragma scalar_storage_order default
```

Absolutely mandatory !

	frame	
byte[0]	CMD	command
byte[1]	A_HH	address
byte[2]	A_H	
byte[3]	A_L	
byte[4]	A_LL	
byte[5]	D_H	data
byte[6]	D_L	
byte[7]	CRC_L	crc
byte[8]	CRC_H	

It’s the solution to mix big with little endian variables !

‘C’ : union – quiz time

What is wrong below ?

- `pixel[2].rgb = pixel[5].rgb;`
- `pixel[0].mixColor = pixel[1].rgb.blue;`
- `pixel[1] = 0x33;`
- `pixel[4].mixColor = 4.75;`
- `pixel[0] = pixel[2];`
- `pixel[3].rgb = 0xFF00FF;`

BITFIELD in C

‘C’ : bitfields

What is a bitfield?

Bitfields offer the ability to minimize to room

- **By the C standard, the compiler is free to store the bit field pretty much in any “random” way it wants. You can never make any assumptions of where the bits are allocated**

```
typedef struct
{
 uint8_t f1:2;              // 2 bits used for this
 uint8_t f2:3;              // 3 bits used for this
 uint8_t f3:3;              // 3 bits used for this
}MyBitield;
```



‘C’ : bitfields

Microchip usage example

R/W-0/0	R/W-0/0	R/W-0/0	R/W-0/0	R/W/HC-0	R/W-0/0	R/W-0/0	R/W-0/0
PSIS	CRS<2:0>			ACLR	MD<2:0>		
bit 7							bit 0

```
typedef union {
      struct {
          unsigned MD                   :3;
          unsigned ADACLR               :1;
          unsigned CRS                  :3;
          unsigned ADPSIS               :1;
      };
      struct {
          unsigned MD0                  :1;
          unsigned MD1                  :1;
          unsigned MD2                  :1;
          unsigned ACLR                 :1;
          unsigned CRS0                 :1;
          unsigned CRS1                 :1;
          unsigned CRS2                 :1;
          unsigned PSIS                 :1;
      };
      struct {
          unsigned ADMD                 :3;
          unsigned                      :1;
          unsigned ADCRS                :3;
      };
  } ADCON2bits_t;
  extern volatile ADCON2bits_t ADCON2bits __at(0xFC0);
```

STM32 HAL don’t use bitfields because compiler compatibility. The mask are preferred.

```
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
  uint32_t odr;
  odr = GPIOx->ODR;     // Get current port state

// Set selected pins that were at low level, and reset ones that were high
GPIOx->BSRR = ((odr & GPIO_Pin) << 16) | (~odr & GPIO_Pin);
}
```

‘C’ : bitfields – quiz time

What is wrong below and how change ADCON2 below?

- `ADCON2bits.MD = (1<<1);`
- `ADCON2bits.MD0 = 0b1;`
- `ADCON2bits.ACLR = 2;`
- `ADCON2bits.CRS = 101;`
- `uint8_t val = ADCON2bits.MD;`
- `ADCON2bits.PSIS = ADCON2bits.MD;`

POINTERS in C

‘C’ : pointers

What is a variable?

A variable is a location in memory with the following characteristics:

- A name (speed, age, …)
- A size (uint8_t, float, …)
- A value (37, 8.412, …)
- Placed at a defined memory address

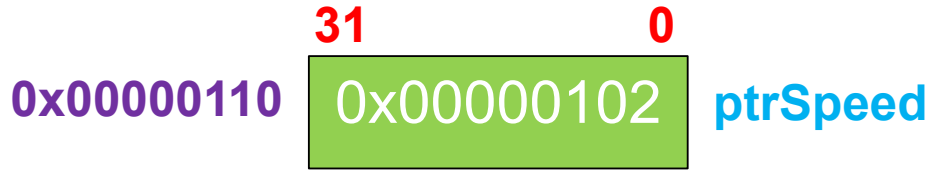


‘C’ : pointers

What is a pointer?

A pointer is a location in memory with the following characteristics:

- **A name (ptrSpeed, …)**
- **A size (depending on architecture, 32 bits for STM32)**
- **A value (0x00000102, …)**
- **Placed at a defined memory address**



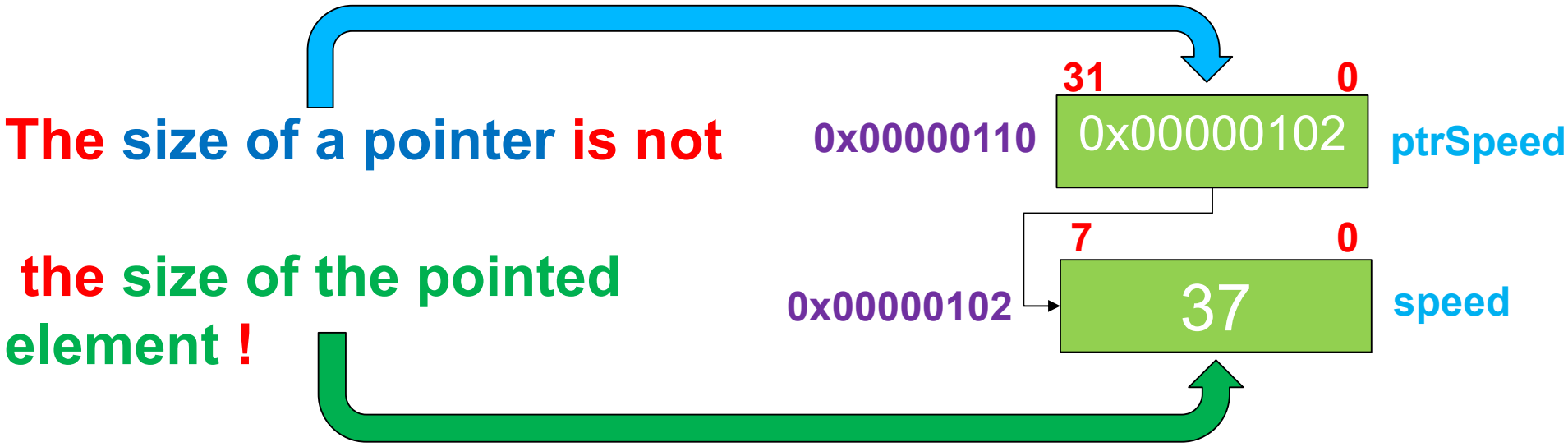
Region description	Address range
Code - external memories	0x0000 0000 0x07FF FFFF
Code - FLASH and SRAM	0x0800 0000 0x0BFF FFFF
	0x0C00 0000 0x0FFF FFFF
Code - external memories	0x1000 0000 0x17FF FFFF
	0x1800 0000 0x1FFF FFFF
SRAM	0x2000 0000 0x2FFF FFFF
	0x3000 0000 0x3FFF FFFF
Peripherals	0x4000 0000 0x4FFF FFFF
	0x5000 0000 0x5FFF FFFF
External memories	0x6000 0000 0xDFFF FFFF

‘C’ : pointers

What is a pointer?

The pointer points to a variable !

```
uint8_t speed = 37;              // the speed variable
uint8_t *ptrSpeed = &speed;      // the pointer pointing to speed
```



‘C’ : pointers
Pointers example

```
uint16_t *pToto;
char *ptrSize;
long * longP;
int16_t* titiPtr;
Struct System *ptrSystem;

int * sizeP, lenP;
```

What is the size of ptrSystem ?

What is the type of lenP ?

‘C’ : pointers

Pointers operators

&

Referencing operator

Returns the address of something

*****

Deferencing operator

Access to the value pointed

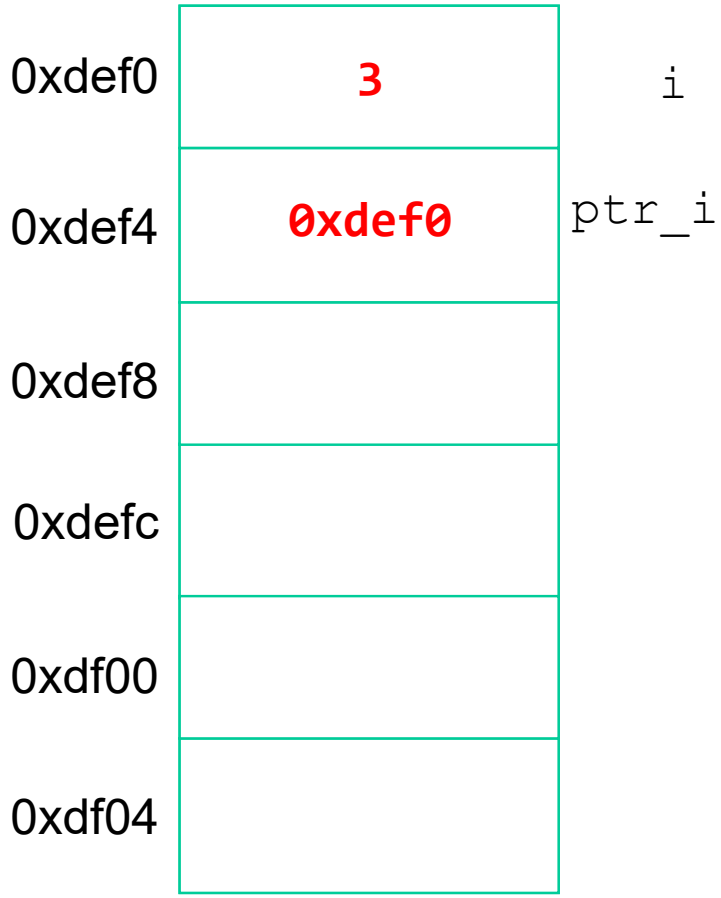
‘C’ : pointers
Example



```
int32_t i=0;
int32_t *ptr_i;

 ptr_i = &i;
 *ptr_i = 3;
```

These two ‘*’ do NOT have the same meaning



‘C’ : pointers
Pointers to constant

```
uint16_t i=20;              // i is a variable
const int16_t *ptr;         // pointer to a constant

  ptr = &i ;                // init pointer address
  *ptr = 33 ;               // modify i ?
```

Error: read only variable is not assignable !



‘C’ : pointers
Constant pointer

```
uint16_t i=20;              // i is a variable
int16_t *const ptr=&i;      // constant pointer initialized

  ptr = &i;                 // init pointer address ?
  *ptr = 33 ;               // modify i
```

Error: cannot assign a constant qualifier !



‘C’ : pointers
Constant pointer to a constant

```
uint16_t i=20;                    // i is a variable
const int16_t *const ptr=&i; // constant pointer initialized

  ptr = &i;                       // init pointer address ?
  *ptr = 33 ;                     // modify i ?
```

Error: cannot assign a constant qualifier !
Error: read only variable is not assignable !



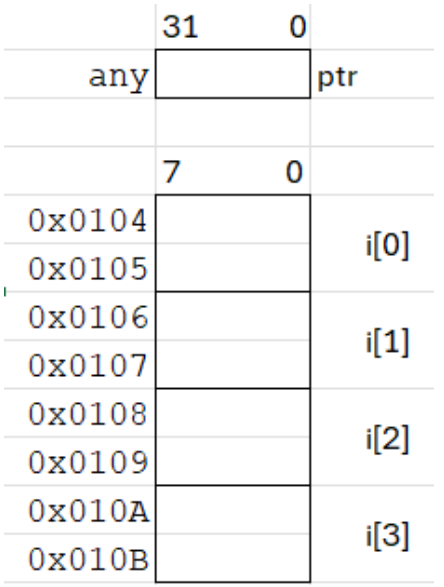
‘C’ : pointers

Caution on pointer type and pointed variable

Pointer can access variable like an array !
Pointer arithmetic is depending on pointed type !

```
void badExample(void)
{
  int16_t i[4];            // i (@ 0x0104, why not)
  int32_t *ptr;            // pointer not initialized

  ptr = &i;                // ptr = 0x104
  *ptr = 10;               // caution, write problem
  ptr[3] = 10;             // caution, double write problem
  ptr++;                   // ptr = ____ ?
}
```



‘C’ : pointers
void pointer

```
void main(void)
{
  int8_t i=0;                 // one 8 bits value
  int16_t j=0;                // one 16 bits value
  void *ptr;                  // void pointer not initialized

  ptr = &i;                   // point to i
  *ptr = 10;                  // error, we don't know access size
  *(int8_t *)ptr = 10;        // typecast is ok
  ptr++;                      // undefined behavior but no error ;-(
}
```

Typecast is mandatory !



‘C’ : pointers

void pointer for function

```
void swap(void *ptr1, void *ptr2, size_t size)
{
  void *ptr = malloc(size);
  memcpy(ptr,ptr1,size);
  memcpy(ptr1,ptr2,size);
  memcpy(ptr2,ptr,size);
  free(ptr);
}
```

```
void main(void)
{
  int16_t x=-5, y = 10;    // two 16 bits value
  int8_t c1='a', c2 ='b'; // two char

  swap(&x,&y,sizeof(int16_t));
  swap(&c1,&c2,sizeof(c1));
}
```

‘C’ : pointers
Exercice 1

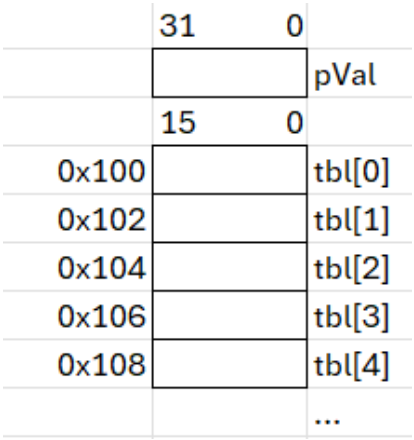
```
void main(void)
{
  int32_t val=27;           // one 32 bits value
  int32_t *pVal;            // one pointer
  int32_t *copyPtr;         // another pointer

  pVal = &var;
  printf("val = %d and %d\n", val, *pVal);
  copyPtr = pVal;
  *copyPtr = 16;
  printf("val = %d and %d\n", val, *pVal);
  pVal = 32;
  printf("val = %d and %d\n", val, *pVal);
}
```

‘C’ : pointers
Exercice 2

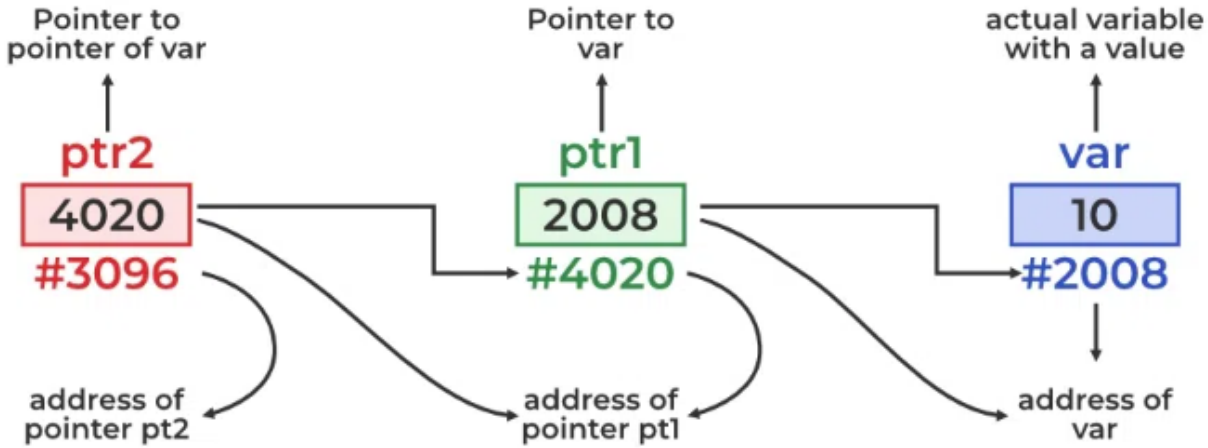
```
void main(void)
{
  int16_t tbl[10];          // one table of 16 bits values
  int16_t *pVal;            // one pointer to 16 bits

  pVal = &tbl;              // pVal = _____________
  *pVal = 124;              // _____________ = 124
  pVal[1] = 85;             // _____________ = 85
  pVal++;                   // pVal = _____________
  *pVal = 7;                // _____________ = 7
  pVal = pVal + 2;          // pVal = _____________
  *pVal = 22;               // _____________ = 22
}
```



‘C’ : pointers

Pointer to pointer




```
void main(void)
{
  uint8_t val = 10;          // one value
  int8_t *ptr1 = &val;       // one pointer to val
  int8_t **ptr2 = &ptr1;     // one pointer to the first pointer

  *ptr1 = 66 + **ptr2;       // what change ?
}
```

‘C’ : pointers

Caution with pointers?

When a pointer is declared, it does not point anywhere
A pointer must point to something with the size of the pointer type

```
uint8_t value;          // one value
uint16_t *ptr;          // bad, at least = NULL

*ptr = 25;              // will be written in unknow 16 bits position
```

```
uint8_t value;          // one value
uint8_t *ptr=&value; // initialised and good type pointed

*ptr = 25;              // value will take 25
```

‘C’ : pointers
Parameter function

```
void invert(uint8_t *numberPtr)
{
  numberPtr = ~*numberPtr;
}

void main(void)
{
  uint8_t number = 33;          // one variable
  ...
  invert(&number);              // pass it as reference
  ...
}
```

Remember struct as parameter (always passed as value)
Remember array as parameter (always passed as reference)

‘C’ : pointers

Quiz time !

Which of the following types correspond to a pointer ?

- **int16_t**
- **double***
- **long**

If I write &toto, what do I obtain ?

- **The address of toto**
- **The value of toto**
- **The value of the variable pointed by toto**

‘C’ : pointers

Quiz time !

Consider the following code:

```
uint32_t number = 8;
uint32_t *pointer = &number;
```

Let’s suppose that `number` is at address `0x01A0` and pointer at address `0x02FC`. If we ask to print *pointer, what will be printed ?

- 0x01A0
- 0x02FC
- 8
- Impossible to predict

‘C’ : pointers

Quiz time !

Consider the following code:

```
uint32_t number = 8;
uint32_t *pointer = &number;
```

Let’s suppose that `number` is at address `0x01A0` and pointer at address `0x2FC`. How to printf the values below ?

- **0x01A0 ->** `printf(“%04X”,  );`
- **0x02FC ->** `printf(“%04X”,  );`
- **8 ->** `printf(“%04d”,  );`

‘C’ : pointers

Quiz time !

Consider the following code:

```
uint32_t *p1 = &a;
```

This correspond to what ?

```
uint32_t *p1;
*p1 = &a;
```

```
uint32_t *p1;
p1 = &a;
```

‘C’ : pointers

Structure pointer

Like a pointer, it points to something

The content access syntax is different

```
Car *ptrCar;
```

‘C’ : pointers

Structure pointer example

```
void main(void)
{
  Car myCar;                        // one car structure
  Car *myCarPtr; = &myCar;          // the pointer to the structure
  myCar.km = 45000;                 // set kilometers
  myCarPtr->maxSpeed = 185;         // maximum speed set with pointer
  myCarIncKm(&myCar);               // function call with reference to structure
}

void myCarIncKm(Car *carPtr)
{
  carPtr->km++;                     // increment kilometers
}
```

-> **Is a structure pointer element access**

‘C’ : pointers

Structure pointer example

```
void main(void)
{
  Car myCar[10];                  // 10 cars table
  Car *myCarPtr; = myCar;         // the pointer to the table
  myCar[1].km = 45000;            // set kilometers for car 1
  myCarPtr[1].maxSpeed = 185;     // maximum speed set with pointer on a table
  myCarIncKm(&myCar[1]);          // function call with reference to structure
}

void myCarIncKm(Car *carPtr)
{
  carPtr->km++;                   // increment kilometers
}
```

. (dot) .maxSpeed has a dot because myCarPtr[1] is a Car, not a *Car

‘C’ : pointers

Function pointer

Like a pointer, it points to something

The function pointer points to a code memory position

This memory position is a function

```
uint8_t (*pf) (uint8_t);
```

‘C’ : pointers
Function pointer example

```
uint32_t triple(uint32_t value)
{
  return value * 3;              // just return 3 * the parameter
}

void main(void)
{
  uint32_t (*pt)(uint32_t) = &triple;     // the function pointer
  uint32_t res = pt(12);                  // call and get result
}
```

`&triple` & is optional !

‘C’ : pointers

Function pointer example (table)

```
void main(void)
{
  int32_t (*ptrArr[])(int32_t, int32_t) = {add, sub, mul};// function tbl
  int32_t ch, a = 15, b = 10;
  while(1)
  {
    printf("Entern choice 0 = add, 1 = sub, 2 = mul\n");
    scanf("%d",&ch);
    int32_t res = (*ptrArr[ch])(a,b);
  }
}
```

‘C’ : pointers

Quiz time !

Which of the following types correspond to function pointer ?

```
1. int(*pf1)(int);
2. double *(*pf2(char *));
3. long (*pf3)(int)(int);
4. void (*pf4)(int,int);
```

Which initialization is right if myFunc is a function ?

```
1. pf1 = &myFunc;
2. pf2 = myFunc;
```

Which call is right if pf1 is a function?

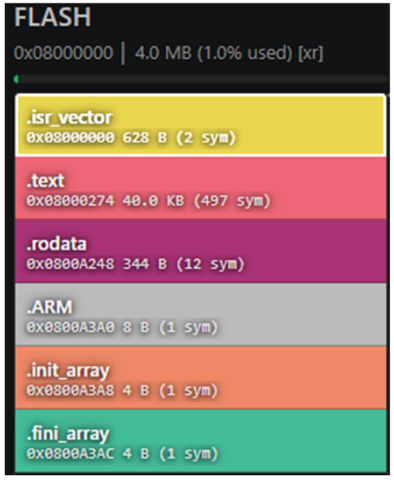
```
1. val = (*pf1)(12);
2. val = (pf1)(12);
3. val = pf1(12);
4. val = (*pf1)12;
5. val = *pf1(12);
```

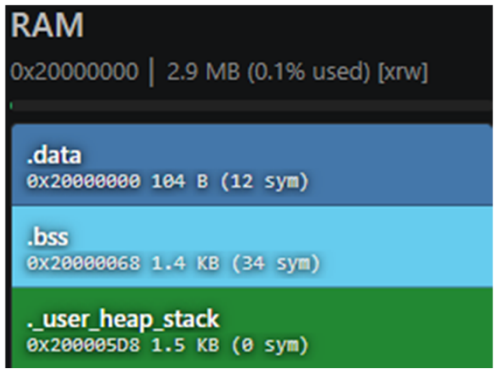
‘C’ : memory allocation

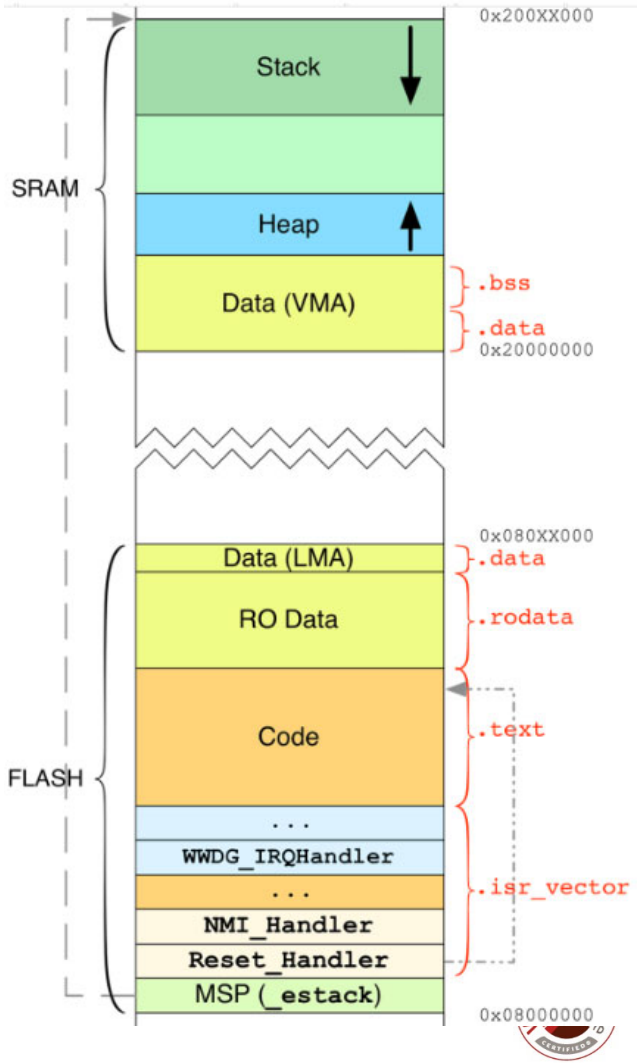
Memory segmentation

How is memory segmented ?

- **stack** is for local variables and parameters
- **bss** is for global variables (forced to 0)
- **data** is for global variables initialized
- **heap** is for dynamic allocation
- **text** is for program instructions







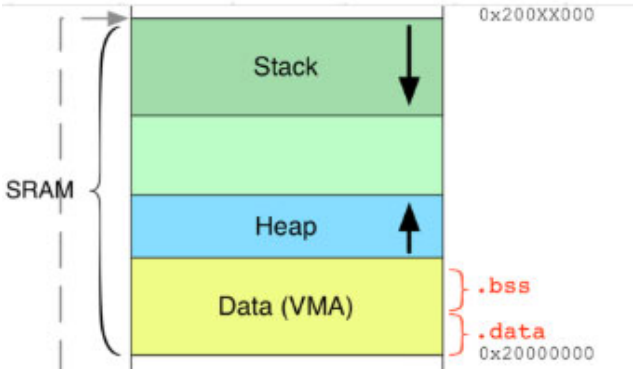
‘C’ : memory allocation

Stack usage

Suppose the function below … do you see a problem ?

```
uint32_t factorial(uint32_t a)
{
  if(a <= 1)
    return 1;
  return (a * factorial(a-1));
}

void main(void)
{
  uint32_t result;                  // factorial result
  result = factorial(20);
}
```

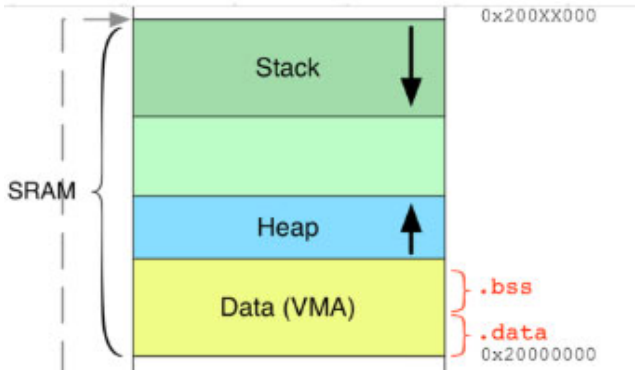


‘C’ : memory allocation

Stack usage

How many bytes used on the stack ?

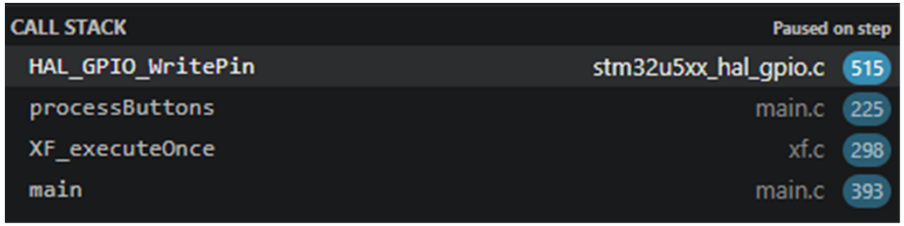
```
uint32_t myFunc(uint8_t a, uint8_t b)
{
  uint8_t tempStr[12];
  for(uint8_t i=0; i<12; i++)
  {
    tempStr[i] = i*a;
  }
  return tempStr[b];
}
```



‘C’ : memory allocation

Stack usage

Dynamic Stack Analyzer view on a breakpoint

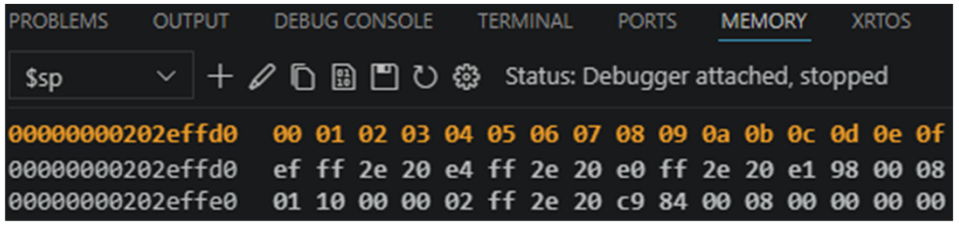


‘C’ : memory allocation

VS Code

Call stack view and stack view





‘C’ : memory allocation

Heap usage

To access the heap, some functions are available in `<stdlib.h>`

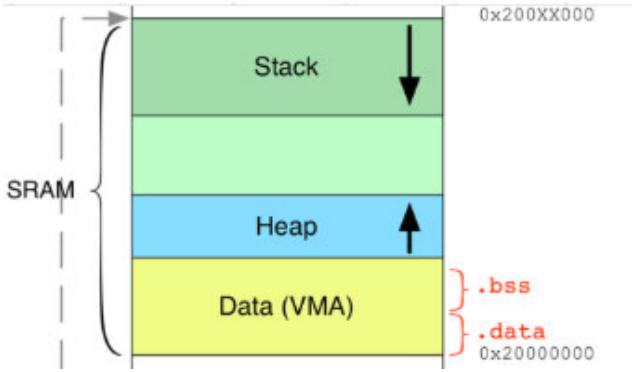
`malloc(), calloc(), realloc(), free()`

```
void * malloc(size_t size);

void * calloc(size_t nitems, size_t size);

void * realloc(void *ptr, size_t size);

void free(void *ptr);
```



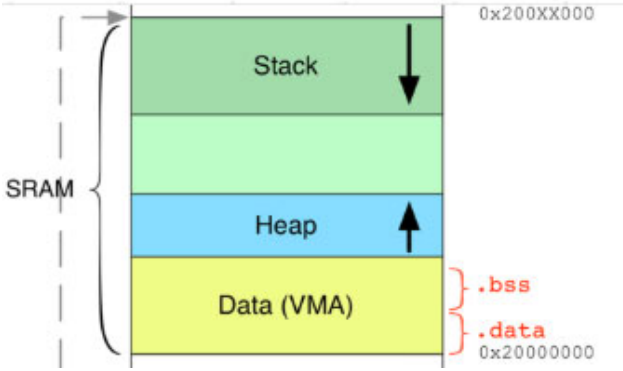
Caution, free does not returns anything !

‘C’ : memory allocation

Heap usage

Understand the example below

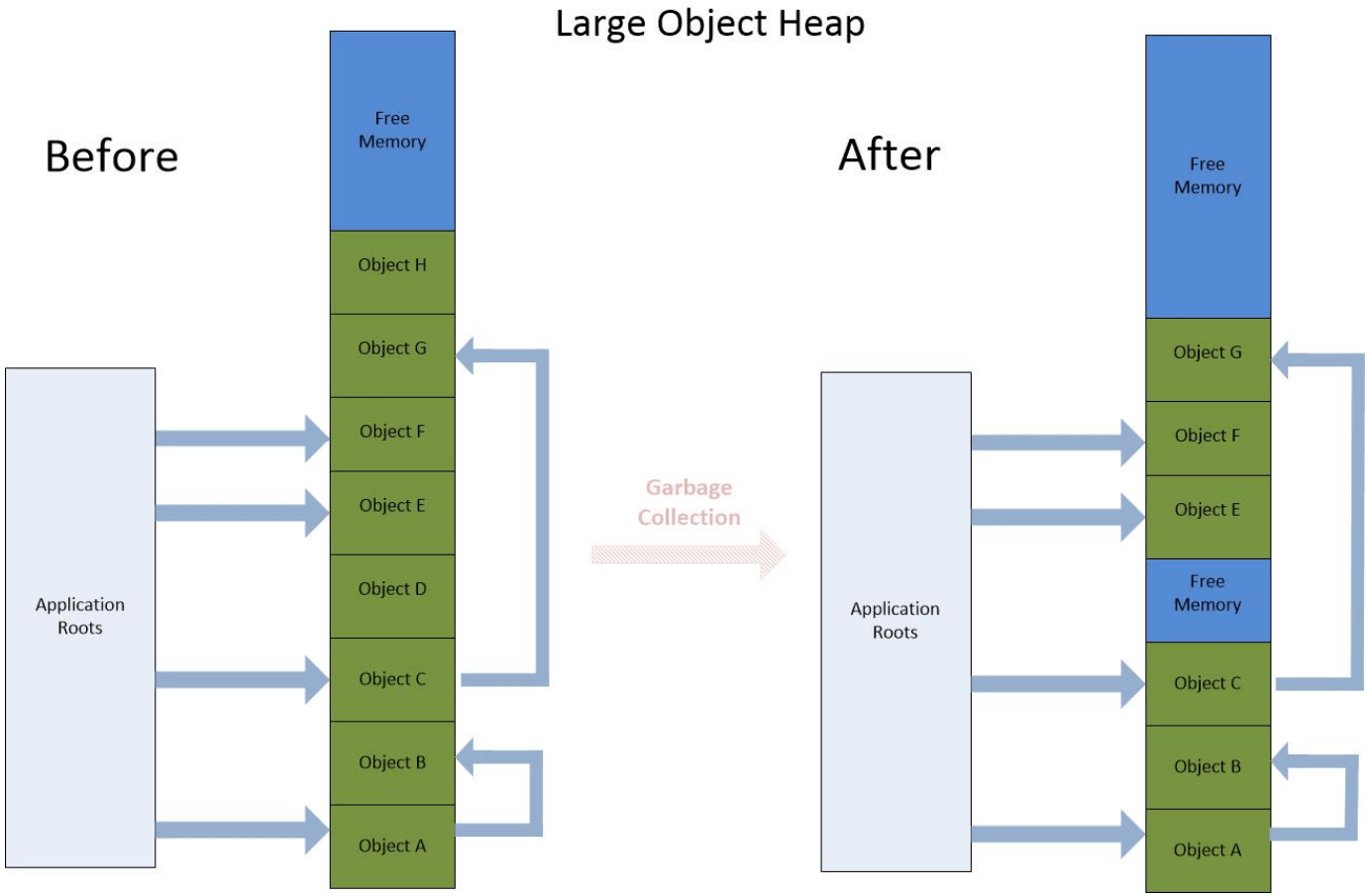
```
void main(void)
{
  uint8_t *memPtr;        // memory pointer
  memPtr = malloc(20);    // get 20 bytes pointer
  *memPtr = 22;           // set 22 to first bytes
  memPtr[1] = 33;         // set 33 to second bytes
  ...
  free(memPtr);           // free space in heap
}
```



All is good or not ?

‘C’ : memory allocation

Heap garbage collection



‘C’ : memory allocation

Heap exercice

Fill the memory as you think if malloc returns the space to the right




```
void main(void)
{
  uint16_t *memPtr;                    // memory pointer
  memPtr = malloc(20);
  *memPtr = 25;
  memPtr[1] = 8;
  memPtr[12] = 66;
  memPtr++;
  *memPtr = 28;
  free(memPtr);
}
```

‘C’ : memory allocation

Heap exercice

A good practice is const pointer

```
void main(void)
{
  uint16_t * const memPtr;  // const memory pointer
  memPtr = malloc(20);
  *memPtr = 25;
  memPtr[1] = 8;
  memPtr[12] = 66;
  memPtr++;
  free(memPtr);
}
```

What will be the compiler result ?

‘C’ : memory allocation

Quiz time !

What will be the size used in stack for these calls?

```
1. void myFunc(uint8_t a, uint32_t b);
2. uint32_t myFunc(struct myStruct a);
3. uint16_t myFunc(uint8_t, table[10]);
```

How many bytes can I use after these calls ?

```
1. myPtr = malloc(40);
2. myPtr = realloc(msPtr,20);
3. myPtr =
   calloc(4,sizeof(myStruct));
```

Comment these lines if myPtr is a 16 bits pointer returned by malloc?

```
1. myPtr++;
2. myPtr[3] = 888;
3. *myPtr = myPtr;
```

‘C’ : memory allocation

STM32U5G9 memory map

```
VARIABLES
Local
  ev = 0x202effe4
  state = STDBY
  oldState = STDBY
  timer1000 = 0 '\000'
WATCH
&state = 0x20000272 <state>
  *&state = STDBY
```

```
GPIOB                    0x42020400
  GPIO_MODER @ 0x0       0x07cfdead
  GPIO_OTYPER @ 0x4      0x00000000
  GPIO_OSPEEDR @ 0x8     0x000000c0
  GPIO_PUPDR @ 0xc       0x50000100
  GPIO_IDR @ 0x10        0x0000c010
  GPIO_ODR @ 0x14        0x00000000

GPIO_ODR@0x42020414
Value: 0x00000000
Reset Value: 0x00000000
```

Region description	Address range
Code - external memories	0x0000 0000 0x07FF FFFF
Code - FLASH and SRAM	0x0800 0000 0x0BFF FFFF
	0x0C00 0000 0x0FFF FFFF
Code - external memories	0x1000 0000 0x17FF FFFF
	0x1800 0000 0x1FFF FFFF
SRAM	0x2000 0000 0x2FFF FFFF
	0x3000 0000 0x3FFF FFFF
Peripherals	0x4000 0000 0x4FFF FFFF
	0x5000 0000 0x5FFF FFFF
External memories	0x6000 0000 0xDFFF FFFF

```
  225:          HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_RESET);
0x08008516              00 22                    movs     r2, #0
0x08008518              40 21                    movs     r1, #64 @ 0x40
0x0800851a              60 48                    ldr      r0, [pc, #384]  @ (0x800869c <processButtons+468>)
0x0800851c              fd f7 82 fd              bl       0x8006024 <HAL_GPIO_WritePin>
```